Journal of Computer Languages 84 (2025) 101342

Contents lists available at ScienceDirect

Journal of Computer Languages

journal homepage: www.elsevier.com/locate/cola

Supporting learners in the transition from block-based to text-based programming, a systematic review

Glenn Strong *, Nina Bresnihan, Brendan Tangney

School of Computer Science and Statistics, Trinity College Dublin, The University of Dublin, Dublin 2, Ireland

ARTICLE INFO

Keywords:
Computer science education
Block-based programming
Text-based programming
Block-to-text transition
Systematic literature review

ABSTRACT

This paper describes a systematic review of the approaches being taken to providing support to learners as they transition from block-based programming environments to text-based ones. It identifies and analyses the literature in the area, identifies the themes which are common across the different approaches being used, and determines gaps in the literature. With the widespread use of block-based programming environments in introductory programming education, the question of how to support learners in the transition to text-based environments has received much attention. The contribution of this paper is to analyse and characterise the approaches being taken to support learners by considering the question: what approaches have been developed to facilitate the transition from block-based programming to text-based programming for learners? To answer this, a systematic literature review was undertaken, combining manual and automatic searches to identify work in the field. A thematic analysis of the literature found eight themes covering technical and non-technical approaches to supporting transition, prompting a set of recommendations for gaps to be addressed in future development in the field.

1. Introduction

The widespread use of block-based programming (BBP) environments to introduce programming to beginners has been one of the great success stories of programming education. The most popular block-based programming environment(BBPE), Scratch [1], has seen a reported 125 million registered users in the 16 years since its first release in 2007. The world of professional programming remains almost entirely committed to text-based programming (TBP), as do almost all programming courses aimed beyond novice users. Because of this students who were introduced to programming through BBPEs must deal with a transition between programming environments that are specifically designed to be easy for novices [2] to those that follow the norms of professional programming (and thus focus on supporting development of sophisticated programs over easy learnability or promotion of engagement). Students will come to this transition at different stages in their development, driven either by the demands of a syllabus, or by personal motivation.

Once students begin to make this transition a number of challenges present themselves, and various approaches have been developed to assist learners in overcoming these transition challenges. There is, however, no clear consensus on the best way to support learners in this, and the diversity of approaches highlights this.

In this work we seek to characterise the current state of understanding of block-to-text transition, and capture what is currently understood as important in supporting learners. We believe that the resulting analysis, and identification of opportunities, will prove useful to the community in the design and development of further efforts in this space.

2. Background and context

Block-based programming environments (BBPEs), and Scratch in particular, are very commonly used with young learners in introductory programming. BBPEs have demonstrated remarkable success in engaging diverse learners by lowering initial barriers to programming and fostering creative expression through code [3,4]. However, while learners can work with BBPEs for some time, sometimes several years, it is normal for them to choose (or to be required) to use more mainstream Text-based programming environments (TBPEs) at some point [5,6]. Many learners find this transition from block-based programming to text-based programming difficult [7]. Assisting learners in making this transition is important both to help transfer technical skills and also to help retain interest in computing.

A number of challenges have been observed as they make this transition [5,8], and there is an ongoing discussion about how best

* Corresponding author.
E-mail addresses: Glenn.Strong@tcd.ie (G. Strong), Nina.Bresnihan@tcd.ie (N. Bresnihan), tangney@tcd.ie (B. Tangney).

https://doi.org/10.1016/j.cola.2025.101342
Received 6 February 2025; Received in revised form 3 June 2025; Accepted 3 June 2025
Available online 18 June 2025

to support this move to text-based programming [6]. Reduction in motivation has been identified as a key factor in students dropping out of CS programmes [9], and the gains in engagement and diversity from the initial use of block-based programming risk being lost during the critical transition to text-based programming if students encounter frustration, diminished self-efficacy, or perceive a disconnect between the creative potential they experienced in BBPEs and the seemingly more technical nature of text-based programming [6,7].

Efforts to support learners in this transition can be motivated by a need to ensure learners transfer specific technical skills they have learned in BBPEs (concepts such as sequencing of instructions, updating of variables, and so on), but they can also be motivated by non-technical goals such as retaining or encouraging the learners' interest in computing. In contexts where computer programming is an optional course of study the latter is sometimes more important than the question of retaining specific skills, particularly if the transition is occurring relatively early in their education [10].

This learning may also be taking place in variety of contexts, meaning that no one approach is universally suitable. BBPEs are used to introduce coding in contexts as diverse as K-12 schooling, self-study, after-school clubs, non-formal voluntary clubs, and even CS101 (i.e. college level) courses, meaning analysis and discussion must take into account a wide range of ages, educational stages, and interests.

Previous reviews in this area have focused on reviewing the introductory environments themselves to obtain a view of what starting points learners have for the introduction of TBPEs [11–16]. The systematic review in [17] is particularly notable in the context of this work as it describes 101 different BBPEs from the perspective of what technical features and affordances they offer, however the review was not limited to those that discuss transition, nor to those that appear only in the peer-reviewed literature. The reviews of BBPEs have consistently highlighted that it is not only the syntactic support offered by BBPEs that is useful. The employment of microworlds, focus on constructivist learning, and playful nature have also been identified as key characteristics. We therefore argue that a review that considers all aspects of how transition is currently being supported is needed to inform the ongoing work in this area.

The contribution of this paper is therefore to analyse the existing peer-reviewed literature on supporting transition from BBPEs to TBPEs in order to determine what approaches are being taken; we are interested not only in identifying the technical affordances of tools but also in drawing in broader questions of supported microworlds, pedagogical support, and affective considerations such as frustration and excitement. By systematically examining effective transition approaches, this review aims to identify strategies that not only preserve technical knowledge but also maintain student interest and engagement — factors crucial for broadening participation in computing education [18].

3. Research question

We are interested not only in what software tools are being developed but in what is informing their design, what interventions are being undertaken in the classroom, what strategies are being using to address difficulties and barriers, and what research is being done into their effectiveness. In the remainder of this review we will use the term 'approaches' for brevity when discussing the range of strategies, interventions, and tools. The term 'environments' will be used for the subset of tools that offer complete systems for writing programs recognising the significance of such systems in this context.

In this review we are interested in examining the literature on all aspects of how learners are supported throughout the transition, seeking to synthesise the evidence of how students can be helped to remain motivated while building on the foundations established by block-based programming. Our research question for investigating the literature is:

RQ: What approaches have been developed to facilitate learners' transition from block-based programming to text-based programming?

To assist in structuring this investigation we sub-divide this into two further questions:

RQ1: What common themes, if any, underlie these approaches?

RQ2: How is evidence for the efficacy of these approaches being established?

4. Review method

A systematic literature review (SLR) is a rigorous and comprehensive method for identifying, evaluating, and synthesising all available research relevant to a specific research question or topic. Unlike traditional narrative reviews, SLRs follow an explicit, reproducible methodology to minimise bias and provide reliable findings upon which conclusions can be drawn and decisions made.

Several established frameworks exist for conducting systematic literature reviews, including the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) [19], the Cochrane Methodology [20], JBI (Joanna Briggs Institute) approach [21], and the EPPI-Centre framework [22]. Each offer unique strengths tailored to specific research contexts. The Cochrane framework is particularly robust for healthcare interventions, while the EPPI-Centre approach excels in education and social policy research with its emphasis on stakeholder involvement.

After careful consideration of various options, PRISMA was deemed most appropriate for our research objectives due to its widespread acceptance across disciplines, clear structural guidance, and transparent reporting standards. The PRISMA framework's 27-item checklist and four-phase flow diagram provided us with explicit direction for documenting the identification, screening, eligibility assessment, and inclusion processes. This methodological clarity was particularly valuable for ensuring reproducibility and minimising potential bias [23].

Fig. 1 shows an adapted flow diagram of the process describing how the records were processed over four stages (identification, screening, eligibility and inclusion) resulting in the final set that are reported on. The identification was itself split into two stages: database searching and snowballing.

4.1. Search strategy

Several methods to establish an initial data set are discussed in the literature [24,25] including conducting a variety of searches, using expert knowledge to establish a good set of known primary studies, forward and backward snowballing, and the use of random sampling with a margin of error in systematic mapping studies [26]. The technique used in this review was most aligned with the quasi-gold standard recommended for reviews [25].

Search terms were defined from the research questions to include articles which address issues of transition from BBP to TBP. An iterative process was used to refine the search queries, with a test set of 15 papers (identified via the authors expertise and manual search of relevant sources) used to validate the results [24].

The search included "transition" and either "visual" or "block-based", and "coding" or "programming", with the specific syntax being adjusted for the database being searched.

Six systematic reviews in related areas were used to identify commonly used databases [11,15,27–30]. The main databases used were IEEE Xplore, ACM Full Text, and Springer Link. The initial search was run in August 2023, with a supplementary search to update results run in June 2024. Database searches produced a total of 180 results.

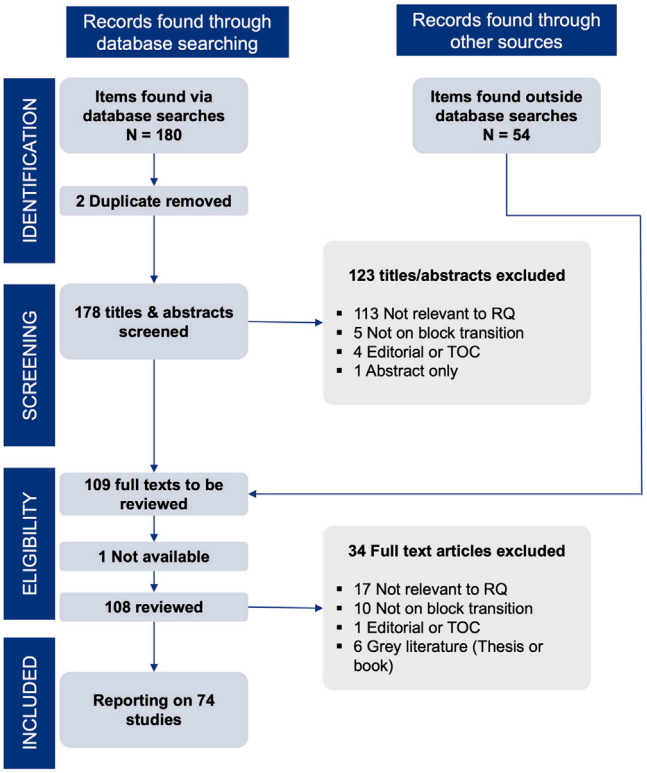


Fig. 1. Prisma flowchart for search process.

4.2. Screening process

Screening was conducted manually with two authors separately screening articles to ensure consistency. The use of machine learning techniques and Large Language models to assist in this process is discussed in the literature [31,32], however in this case manual screening was chosen as the data set was deemed to be of manageable size.

Duplicates were filtered and the remaining articles were screened based on title and abstract. In this phase the titles and abstracts of all 180 articles were retrieved and collected in an Excel sheet, and inclusion criteria were defined. The criteria used were:

- The article must be on the question of programming education
- The article must discuss the question of transition from BBP to TBP
- The article must not be grey literature, a blog, PhD thesis, or an editorial such as a table of contents or an introduction.

The title and abstract was then examined for relevance to the research question. Two articles were screened out as duplicates. 123 articles were screened out as not meeting the inclusion criteria.

4.3. Snowballing

Since our goal was to find as many relevant sources as possible we expanded the search using a snowballing approach, as recommended by Kitchenham [24] and Wohlin [33], who recommend using a hybrid of database searches and snowballing when a search is focused on peer-reviewed literature.

For this study a backwards snowballing strategy was adopted, performed as follows. An Excel sheet was prepared which listed all articles returned by the database search. These articles form the start set of the snowballing stage. The references list for each article was then inspected, and each article judged to be relevant was added to the Excel sheet. The judgement was based by screening on the article title, the context in which it was cited in the article, the article abstract.

Following this round of snowballing each new article added to the list was then treated as the start set for a new round of snowballing; this process was repeated until no new articles were added.

On the first round of snowballing 27 new articles were added to the Excel sheet. A second round of snowballing was then performed on these articles, adding a further 21 articles to the list. A third round of snowballing added six articles; on the final round of snowballing no new articles were identified, and the process concluded, adding 54 articles to the set.

4.4. Review

The remaining set of 109 articles moved to the full text review stage. Each remaining article was imported to the Zotero reference manager for full text screening, and each article was comprehensively read. At this stage inclusion criteria were again applied, eliminating articles which did not discuss block to text transitions, consisted solely of editorial content, or were PhD theses. Books were also excluded at this stage as a matter of practicality.

In total 34 articles were screened out at this stage. 1 article could not be retrieved and was excluded as unavailable, leaving 74 studies for data extraction and coding. All papers included in the review are listed in the bibliography, marked with a † symbol.

4.5. Data extraction, coding, and thematic analysis

The data extraction was conducted by reading again each of the 74 papers in depth. During this process descriptive statistics were extracted and captured in an Excel spreadsheet, noting publication details (year of publication, author, title, publication source). A separate log book was also maintained noting key concepts in each article, and to note connections between articles.

A hybrid approach was taken to coding; our expertise gave an initial set of deductive codes, but the majority of codes were defined inductively during the coding process. A code book was prepared as an Excel sheet, containing short descriptive codes, with accompanying narrative descriptions of each code (to assist in cross-coding), and sample quotations from relevant articles were included to give an "in vivo" sense of how each code is used. As new codes were inductively produced the code book was revised. As each article was read sections of text were annotated in the reference manager with tags corresponding to the codes.

The primary coding was conducted by the first author who also developed the initial code book. A selection of articles was independently coded by the second author using the code book, and a small number of discrepancies were resolved through discussion to produce a consensus code book.

The researchers then discussed and analysed the codes, grouping them to reveal themes. The six-step process described by Braun & Clarke [34] was used to organise this activity. Initial themes were discussed with a group of subject matter experts. The themes were revised with some being renamed, split, or merged until a consensus was reached. The final set of codes and the associated themes are shown in Table 1; deductive codes are italicised.

5. Descriptive statistics

The majority of the papers considered (49) were published at conferences, 23 were articles in journals.

Fig. 2 shows the distribution of year of publication for the papers included in the review. A single outlier from 1988 discussing the DSP visual programming system [35] has been omitted from the figure for clarity. The rise in interest from 2015 onwards is clear and has likely

Table 1
Final codes used in thematic analysis and their organisation by theme.

Theme	Codes assigned to theme
Engagement	Self-expression, *Authentic interests*, Creative, Animation
Evaluation	Evaluation of skills, Evaluation of interest
Pedagogy and curriculum	Syllabus, *Pedagogy*, Game-based learning, Scaffolding, Transfer, paradigm
Perceived authenticity	Authenticity of coding
Satisfaction	Visual feedback, Achieve results quickly, *Achievement (sense of)*
Syntax and naming	*Syntax help*, *Cheat Sheet*, Include learning supports, *Memorisation*, Frustration (negative/experienced), Frustration (positive/avoided)
Translation	Correspondences, *Equivalents*, Allow comparisons, translation (code)
Troubleshooting	Visualisation, Debugging, Preventing Errors, Tracing

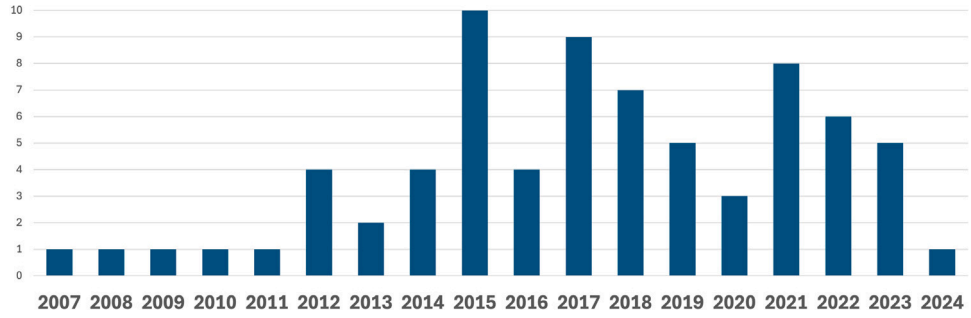


Fig. 2. Distribution of articles by year.

Table 2
Groupings of the themes resulting from thematic analysis.

Thematic group	Themes
Environmental user support	Translation Syntax and naming Troubleshooting
User response, motivation, and engagement	Pedagogy and curriculum Perceived authenticity Satisfaction Engagement
Evaluation	Evaluation approaches to tools

been driven by curricular reforms [36] and broader adoption of block-based programming. In 2015 IEEE hosted the "Blocks and Beyond" workshop focused on BBP and although only two papers from that conference are included in this review it marks a rise in interest in BBP in general, and in the question of how users transition from them to traditional tools.

As well as a general increase in interest we can see an expansion of focus from solely the modality of editing to include explorations of how integrated pedagogical scaffolds can support learners engagement and understanding. A growing emphasis on gathering rigorous empirical evidence in the area can also be seen as more quasi-experimental designs begin appearing in the literature in recent years.

6. Characteristics of tools, interventions, and strategies

This section presents the findings which emerged from the thematic analysis. We have structured the discussion around the following eight major themes: Translation, Syntax and Naming, Troubleshooting, Pedagogy and Curriculum, Perceived Authenticity, Satisfaction, Engagement, and Evaluation.

These themes are further organised into three thematic groups as shown in Table 2

6.1. Environmental user support

The first group of themes consider design features that support learners within the programming environment. Each theme identifies an aspect of how the environments support learners by providing features to support specific aspects of the transition process. Translation considers how learners come to understand that the same programming ideas can be expressed in distinct forms. Syntax and Naming considers the features that help learners become familiar with how to construct programs correctly, and Troubleshooting discusses how learners' understanding is reinforced through correcting errors.

6.1.1. Translation

Helping learners to recognise the correspondences between blocks in BBPEs, and lines of code in TBPEs is discussed in several of the papers reviewed. This involves representing a program in both block and text modes so that the learner can directly see the correspondences.

A variety of approaches have been adapted to show this correspondence (see Table 3). For example (throughout this section we rely on the terminology established by Lin and Weintrop [17]) code may be shown in a "Dual-modality, Shared-view" mode where both versions of the program are displayed simultaneously. This simultaneous view is intended to "emphasize that both notations are just different representations of the same program" [37]. Usually the Block mode can be edited and the text view is updated to match; in some cases both modes can be edited ("Both"), with the Block view being updated when the Text view has been edited in a syntactically correct way.

If the display is not in a Dual-modality, Shared-view style then usually the user can swap between views, seeing only one presentation of the code at a time ("Dual-modality, Distinct view"). An interesting hybrid of these cases is Pencil.cc [36,42], where users can drag and drop blocks into a Text editor window to create new statements (but thereafter must perform edits directly to the text).

In all these cases there is some variety in how literally the Block view of the program maps to the Text view, ranging from exact (where the text on the Blocks is exactly the Text form), through very close representations (which may hide some punctuation details such as braces or semicolons) to approximations where a Block might display a natural language description of the Text representation. Figs. 3 and 4 show typical examples of close (but not quite exact) correspondence in the BlockPy and MUzECS interfaces.

Table 3
Approaches to showing block and text correspondences.

Environment or approach	Display	Block-text proximity	Editing modes
Alice 3	Dual-modality, Shared-view	Somewhat close	Block
App Inventor Java Bridge	One-way transition	Very close	Separated
App Inventor to C	One-way transition	Somewhat close	Separated
Block-C [38]	Block only	Very close	Block
BlockEditor	Dual-modality, Distinct-view	Very close	Both
BlockPy	Dual-modality, Shared-view	Somewhat close	Both
Bricklayer	Dual-modality, Shared-view	Exact	Block
Drawbridge	Dual-modality, Shared-view	Very close	Both
Flip	Dual-modality, Distinct-view	Approximate	Block only
LearnBlock [39]	Dual-modality, Distinct-view	Approximate	Both
MUzECS	Block only	Somewhat close	Block
Patch	Block only	Very close	Block
Pencil.cc	Hybrid	Very close	Hybrid
Pencilcode	Dual-modality, Distinct-view	Exact	Both
Poliglot	Dual-modality, Shared-view	Very close	Both
PyBlockly	Dual-modality, Shared-view	Exact	Block
Scratch to Logo and Python	One-way transition	Somewhat close	Separated
Switch Mode [40,41]	Dual-modality, Distinct-view	Very close	Both
Tiled Grace	Dual-modality, Distinct-view	Exact	Both

Fig. 3. Detail of BlockPy showing somewhat close correspondences.
Source: Adapted from [43].

Not all environments provide a text view, instead programs can be exported to files for use in a standard text editor (and possibly re-imported to produce a new block program) meaning the block and text forms are completely separate creations ("One-way transition").

Finally, we move away from aspects that explicitly and directly support dual representations. The loosest coupling that still emphasises translation has no explicit system support but instead uses entirely disconnected block and text forms — essentially, the learners write programs twice, with a block-only editor and a text-only editor for some entirely different language and have the correspondences between the two highlighted by an instructor. A typical discussion of such an approach appears in [45] which describes how the correspondences between block and text representations is reinforced in classroom activities where students write programs in the (BBP) Alice 3 and then export them to Java, and see "both Alice 3 and Java IDE [...] displayed on a projector and lines are literally drawn from Alice statements to equivalent Java statements" [45]. A similar approach is seen in [46], where learners use the Java Bridge framework, which lessens the burden of writing verbose Java code by implementing a simplified framework, that maps closely on to the App Inventor block language, allowing learners to work in the BBP App Inventor environment and then continue their work in purely TBP Java.

The details of how the user constructs their programs leads to the second of the themes grouped under environmental supports. Having discussed the translation of concepts from one modality to another, we now move on to examine how those concepts are expressed in the concrete syntax of the language.

6.1.2. Syntax and naming

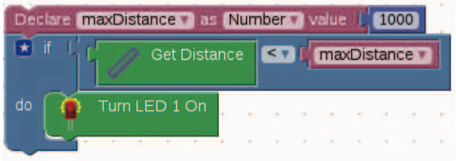
Learners, used to BBPEs become frustrated when they encounter the need to write syntactically correct text and when they discover they need to memorise and recall the names and spellings of keywords, functions, and variables. Almost every paper reviewed recognises the need to provide some kind of support for learners in this regard. Both this and the previous theme are closely tied to the syntactic representation of the program. They consider quite different aspects of the process, however: 'translation' considers how learners are supported in developing an understanding of the meaning of the program by emphasising that there can be more than one representation of the same ideas, while 'syntax and naming' supports learners in the act of constructing specifically the TBP form of the program.

The same editor affordances which were discussed under 'translation', in their role as emphasising that block and text programs can be seen as two representations of the same program structure, can also be used to assist users in overcoming the frustration associated with building syntactically correct programs. The simplest approach (described as "one-way transition" in [17]) allows users to make a transition to standard text editing by exporting the text representation of blocks, allowing them to build a program of some sophistication before they must grapple directly with the syntax etc. of textual code. Some environments allow tighter integration between the two representations by enabling the user to edit the text source in the same environment as the block editor (usually by selecting a control to toggle the view to text mode, as in BrickLayer [47]). A further development of this concept makes it possible to translate the text back to the equivalent block view for further editing in that mode (as in PencilCode [48]. In this last case a decision must be made about how to handle the situation where the text does not represent a correct program, and so there is no block-mode program possible. In Tiled Grace [49,50], the translation feature is disabled until the program is correct, while in Pencilcode [51,52] and BlockPy [43] the program is translated with "placeholder" blocks used to show the incorrect region of the program.

Instead of providing users with a traditional text editor, for manipulating the text-based form of their program, some environments use sophisticated syntax-directed editors that allow the user to edit the text of the program in a constrained fashion such that all programs are necessarily syntactically correct, for example Codestruct [8,53] and Stride [5,54–56]. In effect the user is performing edits on the Abstract Syntax Tree of the program so that the program is either completely correct or is in a clearly understood intermediate state. While evaluations of CodeStruct and Stride have generally shown that the users prefer to have freedom to edit [53,55] they have also found that users sometimes produce programs faster and with less instructor assistance when using a constrained environment [53].

As with the translation issue it is not always necessary to use technical measures to support this and some approaches rely solely, or primarily, on pedagogical support. In [18,57] learners build programs first in Scratch and then build equivalent programs in a TBP language (C# or Python). In [46,58] teaching materials that show side-by-side correspondences between block and text forms are used to support the transition from AppInventor (a BBPE) to Java.

Having considered these scaffolding features that assist in the creation of the program, we then consider the fact that the program may end up being incorrect and how environments can provide feedback to help the learner correct it.




```
void setup()
{
  maxDistance = 10000;

  Serial.begin(9600);

  pinMode(3, OUTPUT);

  pinMode(2, OUTPUT);

  pinMode(5, OUTPUT);
}

void loop()
{
  if (smoothDistance() <= maxDistance) {
    digitalWrite(5, HIGH);
  }
```

Fig. 4. Detail of MUzECS showing somewhat close correspondences.
Source: Adapted from [44].

6.1.3. Troubleshooting

The importance of having learners experiment and debug with their programs to improve understanding is widely recognised. Scratch allows users to create quite sophisticated programs without encountering runtime errors, and when learners begin the transition to TBP they have to learn new troubleshooting skills with the result that it now seems to them that it takes more work to achieve the same results [59,60]. To put it another way: "*Scratch may raise the threshold to transfer to textual programming: achievements may remain frustratingly modest due to frequent errors*" [61]. A belief in the need for adequate tools to help learners reason about their programs and develop some understanding of the 'notional machine', as Sorva [62] terms it, that they are working with can be seen in many of the environments reported on in the literature. These tools can take many forms.

Attempts to highlight errors before the program is run are discussed often, the intent being to reduce the need to debug at run time (as the authors of Tiled Grace put it, they "would prefer to show errors at the time they are made, or even to prevent their occurrence altogether" [50]). Usually this relies on the kind of syntax-directed editing discussed under "translation". Since the program structure is understood by the editor it is possible to highlight or block the introduction of some classes of errors [44,49,63,64].

Integrating error reporting into the editing process helps the user uncover mistakes during the writing process instead of by later debugging of the running program. Generally this means highlighting incorrect code and offering error messages as pop-up dialogs, as in Tiled Grace [49,50] and Stride [56]. The sorts of errors that can be identified vary, but examples include: use of undefined or uninitialised variables (as in [44]), use of variable names outside their correct scope [49], type errors [50], and in some cases structural errors, as in [44] where Arduino programs which are missing the required "loop" structure, or in [47] where redundant initialisation of devices is identified and flagged to the user.

Another approach taken by tools is to assist learners in investigating the meaning of their program.

Visual techniques can help the learners investigate and debug their program by understanding how sections of the program relate to each other. The most common practice is to overlay the editing experience with visual indicators of elements such as scope and indentation [49,50,56,64,65]. Tiled Grace [49,50] takes this approach further using an overlay mechanism to render "out of band" information — the environment can overlay highlights and draw lines connecting variable declarations and usage. Visual Logic [66] attempts something similar for the flow of execution by using a flowchart based program editor that necessarily provides a visualisation of the program control flow; although the mapping to the statements of a structured programming language such as Python or Java is imperfect. DesignScript [67] takes a similar data flow model (targeting C). MUzECS [44] takes advantage of its very focused application domain of microprocessors by placing graphics, such as small LED speaker icons, on blocks that control devices, as a simple mechanism to visualise the meaning of the block.

No matter how much guidance is given it is inevitable that errors will occur in the running program. We therefore find tool support for explicit debugging is also evident in this theme. Niemela [61] recommends that "debugging tools should be as informative as possible" [61]. The most common form of this discussed in the literature is step-by-step program tracing where execution proceeds one statement at a time with pauses (either timed or until the user clicks a button) between statements. This feature is present in Scratch and some other BBPEs such as Makecode, and so learners may be familiar with it already. Patch [68] emphasises its use for troubleshooting, and in [69] the authors explicitly note that "step by step execution […] was the most referenced feature among all the features referenced by students as important".

Even where program tracing is not directly supported its utility is widely acknowledged (for example Alrubaye et al. [70] used a hand-tracing approach in their student evaluations). Mecca et al. [71] recorded traces of program execution which are later used to build animated 3D simulations of program execution that learners can view to develop an understanding of their 'notional machine'. The authors note the utility of this for students trying to debug their work and go on to suggest that adding further features from professional debuggers may be beneficial.

6.2. User response, motivation, and engagement

This second group of themes address the learner-centred and pedagogical aspects of transition. These themes shape the educational context for the learner: "Pedagogy and Curriculum" discusses strategies that are adopted to scaffold essential programming concepts. "Perceived Authenticity" discusses the need to respond to the learners perceptions of what constitutes 'real programming'. The "Satisfaction" theme emphasises the role of early successes and positive reinforcement, while "Engagement" discusses the value of connecting programming tasks to the learners' personal interests.

6.2.1. Pedagogy and curriculum

Some of the reported interventions rely solely on pedagogy [10,18,57,69] and do not provide any technical scaffolding or support to aid in the transition. Under this theme we discuss these approaches.

Recognition of the need to consider the pedagogical approach being used appears frequently in the literature reviewed [7,11,17,38,61,68,72–74]. In [42] a number of approaches are evaluated, concluding that BBPEs work well in conjunction with TBP languages and proposing a pathway and syllabus to support this. Despite this recognition relatively few authors (e.g. [18,45,70,75]) actually make explicit their proposed pedagogical models.

A number of common pedagogy-related concepts do emerge from the literature. In particular the need for, and utility of, providing suitable supports that will be gradually removed ('scaffolding' and 'fading') is widely recognised [8,38,49,61,76–78]. In some cases there

are highly managed experiences as in Pencilcode [51] where beginners are taken to the 'Pencilcode Gym' before the full coding environment is available, or the 'dialects' of Grace [49,50] where features of the language can be disabled to simplify the process. In others, such as Kazemitabaar et al. [8] the scaffolding is achieved by having users begin work in a specialised environment then removing the supports carefully as they move to a purely text-based environment. A scaffolded approach is sometimes reflected purely by the choice of syllabus design (e.g. [7,10,18,57,75]) without requiring an entirely new programming environment to be created.

As well as explicitly pedagogical techniques a recognition of the need to cover specific components of the curriculum is often discussed, in particular the need to cover basic foundations of computer programming [7,8,38,56,57,61,70,79,80]. Some authors note a need to align with national curricula [68] requiring additional concepts to be included. Tension between keeping the syllabus simple and developing games and other content that engages users interest (see Section 6.2.2) which can require some advanced programming concepts is noted by two of the studies [18,46].

There also is discussion focused on the need for teaching strategies promoting transfer of specific programming skills [5,7,8,43,49,51,58,70,75,81] from BBPEs to TBPEs. Some authors point out that there are difficulties of establishing that transfer is taking place [8,18] with some studies showing that transfer can be limited [73] or even that the wrong approach in promoting transition can amplify the challenges students face [7,65].

Using a pedagogical approach to tackle the particular transition challenge of a change in programming paradigm also emerged. Many BBPEs offer concurrent or event-driven programming models, while most novice TBP is sequential and procedural. This change in programming paradigm is identified as a challenge and there is some discussion, either pointing out the difficulty [6,68] or attempting to mitigate it by working with event-driven programming styles [39,75,82,83]. In [7] the authors note that the lack of a true OO model in Alice causes difficulties in further transfer to languages such as C++ suggesting this could be a further point of potential later confusion if not handled carefully.

Building on the sense of how learning materials are presented the next three themes discuss how the learners respond to broader patterns of motivation and persistence.

6.2.2. Perceived authenticity

A recurring observation in the literature is that students perceive BBPEs as being "not real" programming. This perception of inauthenticity can lead students to claim they have never programmed a computer before even when they have had previous experience of a BBPE [6,78]. This perception of inauthenticity then translates into a lack of confidence when students move on to text-based programming [50]. The perception that their BBP experience has not prepared them for working in a TBPE can give rise to frustration that they had to "start over" in a TBP language after studying programming in BBP modes [80].

Learners have been questioned on their perceptions after moving from blocks to text. In [61] we find learners saying while they are happy to learn Scratch they see it as important they "should learn JS [JavaScript], as that's more like real coding"; Weintrop and Wilensky [84] report similar comments. In [58] learners who have built mobile applications in AppInventor are reported as demanding a more professional way of programming. Several papers reporting on experiences with the Alice BBPE identify similar concerns [7,65,69,80] a representative quote summarises it thus: "Alice was all about dragging and dropping lines of code, and it wasn't until I learned Java, that I actually understood what the code is and what it does" [65]. A rare dissent from this is in the reporting on the evaluation of Stride [56], where the authors specifically note that any such perceptions left no evidence of affecting their students satisfaction (although they did not directly investigate perceptions of authenticity).

This student perception has motivated designers of programming environments supporting transition to include specific features intended to make languages feel more "authentic" to users, such as the inclusion of web development idioms in PencilCode [51]. This perception is also identified as an additional motivation for reinforcing the equivalence of block and text code (as in [50] where the display of the users code can be freely toggled between block and text representations), but it is more common to simply note it, sometimes with an observation that it actually motivates the move to TBP [8,11,17,18,36,70,82].

Noting the influence of this sense of authenticity leads us to consider another aspect of the learners affective reaction to programming education: the sense of satisfaction or achievement in what they have made.

6.2.3. Satisfaction

The literature frequently identifies a need to give learners an early sense of satisfaction in order to retain their interest in programming. This has influenced the design of several systems in two key ways: giving students the ability to create graphical output from their programs, and simplifying the path to getting these programs running so that students achieve results quickly.

It is widely noted that students generally respond well to producing programs with graphics. Krpan et al. [18] notes that students appreciate "teaching strategies with visual incentive". A variety of approaches are found in the literature to support this. The most common one is via some form of 'turtle graphics' support as pioneered in Logo. This approach is used very widely: Scratch, PencilCode and its variants [8,36,51,79], Patch [68], Visual Logic [66,85,86], Block Python [82], Tiled Grace [49,50], and Greenfoot [11] all provide turtle graphics interfaces that allow users to generate graphical output in a simple way.

More complex support for creating sophisticated animations and games has also been tried. Alice [7,65] supports "graphical programs that manipulate 3-D objects in a 3-D virtual world" [7] allowing students to create animations and games. Producing such results in a traditional programming language would require considerable effort before any results could be seen, but the Alice environment makes it easy to manipulate the objects directly and get pleasing results with a minimum of code, thus keeping students engaged as the Alice environment introduces TBP.

The complexity of working with 3-D objects means it is more usual to find environments supporting 2-D sprite-style graphics of the sort found in Scratch which are easier for learners to manipulate. Patch [68], based on Scratch takes this approach. Something similar is visible in Greenfoot [5] which visualises object-based actors on a stage. In [18] the authors use Scratch and then Snap, using Sprites in both to help students develop a concrete understanding of their programs' behaviour (noting that "Many general-purpose programming language constructs are too abstract for novices" without visual feedback). They then move learners to C# using a custom framework that supports similar Sprite features in an attempt to help students achieve encouraging results quickly.

Some other approaches have been tried by a small number of environments; Flip [63] allows users to quickly create descriptions of games using a simple flowchart-like editor which are then executed using the commercial Neverwinter Nights 2 engine. This gives the users a sophisticated interactive 3D environment in which to play out their designs. The easily accessible entry point that the Neverwinter Nights 2 editor initially offers (allowing users to create scenes and characters in the game easily) is highlighted as being "invaluable for motivation" [63].

Continuing the idea of fostering a sense of achievement, in [75] the authors take an approach to creating mobile apps that minimises the amount of coding required of a student before they can produce a useful app, noting that "Using a proper visual tool, such as the App Inventor, gives students a great opportunity to create attractive and

useful applications with ease, so they experience the feel[ing] of success immediately" [75]. Students are then scaffolded into using a Java 'App Inventor Bridge' library to continue development in the TBPE.

Taking a different approach, but with a similar emphasis on helping students understand the abstract behaviour of their programs through visualisation, the Diogene-CT [71] system visualises the behaviour of the learner's program using a 3D robotics simulation of a computer, allowing learners to see a very concrete simulation of their program. While not facilitating the same creation of games and animations as other systems the visual feedback is still intended to give learners satisfaction in seeing their program run. Physical computing with real hardware (usually Arduino) occasionally appears in the literature reviewed [39,44,47,87,88].

Finally, Drawbridge [78] offers a very immediate way for users to create animations and games by allowing users to capture their own drawings on paper, create animations by example (dragging objects on a canvas), and then adding more complexity as needed using code. Simplifying the process of creating the initial animations "provided major motivation to participants" [78].

The overall sense presented in the literature reviewed is that by achieving results quickly by providing immediate visual feedback gives learners a sense of satisfaction that helps them remain motivated as the complexity of the programs they are attempting to write grows.

6.2.4. Engagement

Promoting student engagement appears frequently as a concern.

BBPEs generally support creating engaging multimedia content. Moors et. Al [6] note potential downsides to this positive experience as students move to TBPEs. Learners become used to the being able to create games and animations without a great deal of effort in BBPEs. Students may not see the purpose of switching to TBPEs that require much more effort to achieve similar results. Both Moors et. Al. [6] and Krpan et al. [18] also note that there is a disconnect between the interests of students and the typical content of introductory programming courses based on TBPEs, which often feature a strong emphasis on numeric and symbolic applications rather than "real-world" applications.

The conclusion many authors have come to is that environments focused on supporting transition should give good support for letting students create programs that engage with their interests, and do so as simply as possible. We see this widely reflected in the literature.

For some students these authentic interests may well be computing itself, but many students are motivated by other interests. In a case study on learner motivation [61] found that "Intrinsically motivated students enjoy programming, whereas students with other motivations need external triggers to ignite interest". The second group are described as having "authentic goals" which computing can support (for example, "[our] chemistry teacher had us make simulations of scientific experiments, which I think helped people see how their programming knowledge can be made useful in real life situations" [61]).

Creating mobile apps are one such authentic interest that has been explored in the literature, generally through the use of App Inventor [46,69,75]. Writing on Alice frequently reports that storytelling is considered another "intrinsically motivating activity" [7]; and it is encouraged in some other systems such as Pencilcode (which supports "music and storytelling" [51,70]). Support for creating animations is also noted as important; in their systematic review of visual and textual programming languages Noone and Mooney recognise that "Many young learners will watch animated shows or play games, and, from our experience, they love the feeling of seeing their own ideas and animations come to fruition" [11].

By far the most common authentic interest reported is authoring of games. Creation of games is noted frequently as a mechanism to engage student interest, from Alice [6,7,65,75] to App Inventor [89], Greenfoot [56], Bricklayer [47] all describing the use of game development to promote student interest. Some environments embed this interest very deeply in their design; Flip [63] is oriented completely around the goal of producing scenarios for the commercial role-playing game "Neverwinter Nights", while MiniScript [90] is a small teaching-focused language optimised for embedding in games.

In general enabling "creative self-expression" [7] in students is seen as very positive, and strong engagement is often seen (for example, with Flip teachers reported seeing "some pupils coming in at lunch breaks to work on their games, and continuing to come to the computer lab to put finishing touches on their work even after the course had ended". Howland and Good [63], and in one Alice based intervention an author "witnessed Alice captivate an 11-year old female in a matter of hours, prompting her to spend days creating a full game" [80]). From an inclusion perspective it is worth noting that "it may be that embedding programming within a narrative-based activity makes it both more interesting, and more relevant to girls" [63].

Taken as a whole, these four themes suggest key supportive design decisions that can be taken to meaningfully influence learners persistence, motivation, and engagement.

6.3. Evaluation

The final theme examines how empirical studies are used to assess the effectiveness of the approaches; we outline the methodologies and techniques in order to give a critical perspective on how these environments have been evaluated.

6.3.1. Evaluation approaches to tools and interventions

These four themes have discussed how users respond to, are motivated by, or engage with programming in the context of the transition from block to text programming. One final theme was identified in the literature which we now come to. Across the 74 papers we found only 34 reported evaluation studies of one sort or another. Of the 34 reporting evaluation 4 were found not to be reporting results (only that trials had been conducted, but not the results of them), or were reporting on data also covered in other included papers. This left 34 papers to be examined.

In 5 cases the results were not presented in enough detail to comment on, or the results were too preliminary, or not in sufficient depth for commentary. We now summarise the modes of evaluation reported in the remaining 29 papers. Two papers [71,79] reported on multiple studies and so for clarity in the remainder of this section we report on the 31 studies rather than 29 papers.

Cohort sizes varied widely — the mean size of the cohort was 42, but with a standard deviation of 33. One study [71] did not report on cohort size. Fig. 5 shows the distribution of cohort sizes. Nearly half of the studies reporting cohort sizes (15 of the 31) had fewer than 30 individuals involved. Of those reporting larger cohorts nearly half (7 of 31) had 80 or more individuals.

The duration of the interventions was more evenly distributed (see Fig. 6), with 10 studies reporting durations of one week or less (mostly single sessions), 6 reporting up to five week durations, and eleven reporting on longer interventions (usually one school term, although an outlier is [71] which reported on a study that ran for four years). One study [61] did not report on the duration. Specific details of number of classroom hours were not generally reported.

Generally, studies involving larger cohorts (more than 80 individuals) also tended to have involved longer interventions with 5 (of 7) such studies being conducted over more than five weeks.

We analysed the literature seeking to identify patterns across studies. While the majority of studies collected data on students skills and attitudes we did note that some studies were closer to usability studies of specific tools, and that these studies were mainly, though not exclusively, very short in duration (one to three sessions). However, many experiments measure some combination of instructional efficacy, affective responses, and tool usability. This emphasises recognition of the need to combine these aspects for a successful approach.

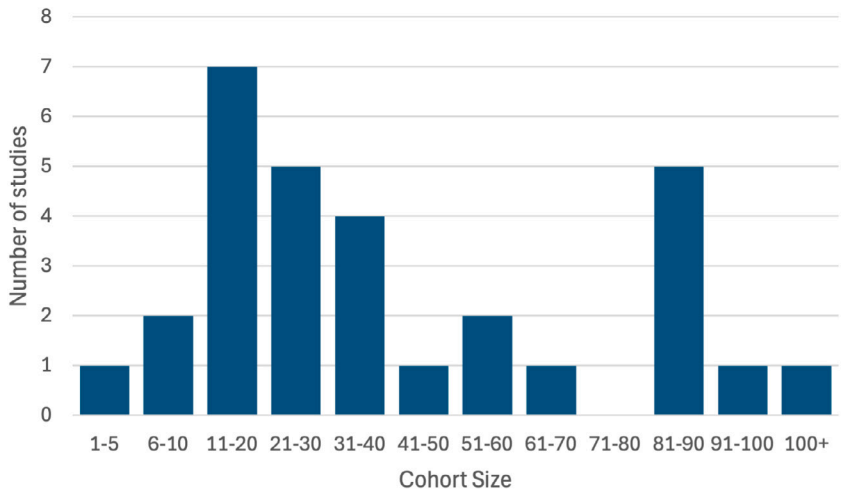


Fig. 5. Frequency of cohort sizes.

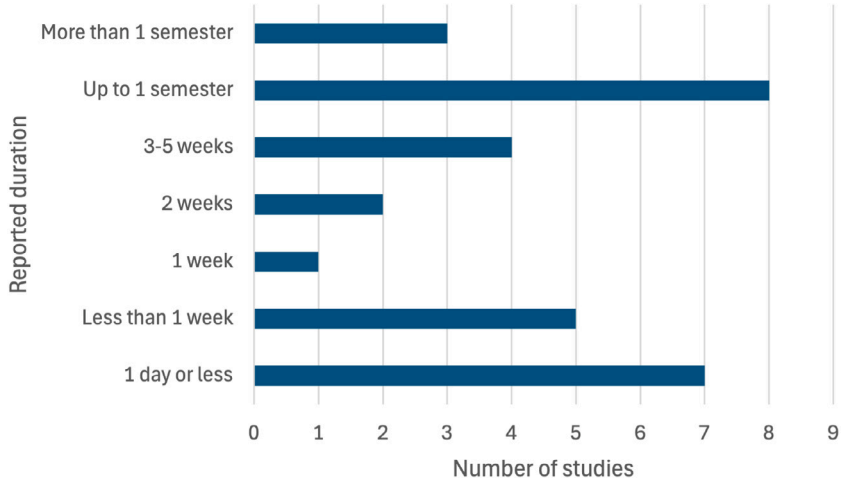


Fig. 6. Reported duration of study.

In general, there were two sorts of data collected. First, evaluations of participants programming skills are common [8,10,36,38,42,46,55,58,70,71,73,76,78,91], either by an inspection of the artefacts they constructed or by having them complete a test after the intervention. Second are investigations of participants' opinions and attitudes to programming, which includes questions of self-efficacy, confidence, and motivation and interest in continuing to learn programming [10,18,36,50,56,61,63,65,69,71,79,92–94]. Attitudes are evaluated either via surveys (usually pre- and post-intervention) or by participant interviews. Two studies [95,96] evaluated differences in cognitive function. One paper [97] examined the impact of usability issues in the Edublocks interface.

Measurement of skills and attitudes is evenly split among the papers reviewed (18 studies report gathering data on skills while 17 report gathering data on attitudes. Only 6 gathered data on both from the same cohorts).

Overall the finding is that while evaluation is important it is not always performed in depth, with many evaluation reports only covering a single session, or a small number of classes lasting less than a week. There are, of course, exceptions where some tools have been evaluated over a longer period of time, and we would encourage more of such in-depth evaluations to strengthen the evidence base for what approaches are successful.

7. Discussion

The original research question asked "What strategies, interventions, and tools have been developed to facilitate learners' transition from block-based programming to text-based programming?". Sub-question RQ1 ("What common themes, if any, underlie these approaches?") was addressed in Section 6, and sub-question RQ2 ("How is evidence for the efficacy of these approaches being established?") was addressed in Section 6.3.1. In this section we summarise the findings as they relate to our main research question, and present some observations and recommendations arising from our review.[1]

7.1. Appropriate technical and non-technical supports

There are two complementary approaches to supporting learners in the transition from BBP to TBP that have been developed, one built on technological supports and emphasising an understanding of the underlying structure of programs, syntax, and representation (Sections 6.1.1–6.1.3), and the second primarily concerned with how learners react to the tasks and knowledge (Sections 6.2.1–6.2.4). In

[1] One paper by the first author which was produced as a response to analysis was excluded from the discussion.

the first we find a number of technical approaches to supporting transition. From the three relevant themes that emerged from our analysis, Translation, Syntax and Naming, and Troubleshooting, we found some common themes in the design of these environments: (1) a belief that it is important to highlight how the underlying meaning of a program can be thought of separately to the modality (block or text) of how the program is expressed, (2) a belief that learners benefit from support when creating programs in text, both to construct syntax correctly and to recall the terms (names and syntactic structures) are available, and (3) a belief that as programs become more complex learners need more troubleshooting support (either via source code visualisation techniques or via run-time supports for debugging).

The second approach focuses on ensuring motivation and engagement are kept high as the transition is underway, emphasising a need to give rapid satisfactory outcomes and maintaining a sense of authenticity in the experience itself (contrasting this with the perceived inauthenticity of BBPEs) while engaging with problem domains and situations that are genuinely interesting and engaging for learners. The common themes that emerge in the literature reviewed relating to this are: the importance of appropriate pedagogy and curriculum, the perceived authenticity of the tools used, giving learners an early sense of satisfaction in their achievements, and encouraging engagement through the learners own interests.

The analysis presented in this study strongly supports the argument that technical approaches – such as dual-modality interfaces and troubleshooting support – and scaffolded curricula must be developed in tandem, so that each component reinforces the other.

While it is clear that supporting learners in the transition from BBP to TBP is a complex challenge with no single solution the identification of these two broad approaches leads us to the first recommendation for future work:

Recommendation 1: There is a need to investigate the most effective approaches for combining appropriate technical and pedagogical supports so that they complement each other in the learning process. Future approaches should ensure both approaches are appropriately considered.

7.2. Preserving the familiar

The themes of Translation and Syntax and Naming reflect in part attempts to help learners tie their existing knowledge of BBP to their experiences in TBP. Reducing the amount of "unlearning" that must be gone through is a valuable goal in itself, however we find repeated evidence that environments that help the learner map block structures to textual code also improve conceptual transfer and assist in a smooth transition [17,36,37,53].

The discussion in the literature reviewed which we have identified as the themes of Satisfaction and Engagement (Sections 6.2.3 and 6.2.4) suggests that it may also be valuable to continue to support the use of interactive multimedia programs to help promote motivation. There is an implicit connection in the literature between giving learners a sense of satisfaction and achievement by having them produce visually engaging outputs (games, animations, mobile apps, and so on) and the idea that engaging with authentic interests and motivations through creative outputs help to make the experience of programming feel authentic and satisfying. We can see the potential here to develop a connection to the sense of "authenticity" (in the sense of engaging with the learners perception that TBP is "real" programming), and an opportunity to cultivate a sense that "real" programming can be a rewarding and satisfying experience. In practice, however, this can be challenging as the programming style required in most TBPEs for this kind of development is quite different to that supported by most BBPEs.

A gap seems to exist around ways to retain learners' knowledge of event-driven and concurrent programming paradigms in the transition away from BBPEs, where such models are common. We might say that the transition to text can also become a transition from one style of programming to another. There is a connection to the emphasis on enabling learners to create games and animations that appears in the literature as a source of motivation and self-expression, game development often relies on event-driven architectures that can be hard to introduce in TBP languages in a simple way — as Moors et al. [6] put it, "Creating a game or animation in a textual language requires much more effort than in Scratch or Alice". Dolgopolovas et al. [89] observe that "App Inventor is based on an event driven paradigm that is the opposite of 'classical' structured C". Investigating ways to avoid an abrupt paradigm shift in these domains, as is attempted in App Inventor Bridge, could be a useful avenue of investigation. This leads us to our next two recommendations for future work:

Recommendation 2: Giving learners a sense that they are engaged in meaningful work is important to their motivation. Further work should be done to investigate how approaches to transition can ensure that the tasks learners can engage in are relevant to them, and that the tools provided give learners a sense that they are developing skills that will be meaningful outside of a purely educational context.

Recommendation 3: Supporting recommendation 2, the question of how learners could continue to make use of the program design approaches they have already become familiar with in BBPEs should be investigated

Returning to the question of helping learners associate their previous learning with new ways to express programs we note that providing mechanisms to explicitly show the correspondences between block and text representations is a common focus, especially when specialised technical environments are used, but that there is wide variety in how this is implemented. Some approaches present the learner with a very direct side by side view of the block and text versions of the same program ("Dual-modality, Shared-view"), or allow dynamic toggling between two views ("Dual-modality, Distinct-view"). It is not uncommon for one mode to be considered the "primary representation" of the program (Noone & Mooney [11], for example, describe the text representation of programs this way), although we would argue that it may be more helpful for both representations to properly be viewed as projections of an abstract syntax tree (AST) [98]. Others provide less literal translations featuring a range of approximations going as far as diagrams and natural language. There is a clear consensus that learners should be helped to see these correspondences but how best to structure this remains an open question. This leads us to the next recommendation:

Recommendation 4: Highlighting the conceptual correspondences between different representations (block and text) of the same program can be an effective strategy for supporting transfer, more work should be done to determine which approaches best facilitate this.

7.3. Support troubleshooting and interactive investigation

Efforts to highlight errors early in the development process have been successful, giving learners feedback on basic syntax errors, type errors, and so on helps avoid later frustration and if error reports can be presented as early as possible. Nevertheless, as learners move to developing more complex programs the risk of logic errors in the final programs rises, and some thought must be given to how learners can debug their programs. Striking the right balance between providing supports that help learners investigate how their programs are working and keeping environments from overwhelming users requires some careful thought, and the literature reviewed here does not come to a clear consensus. There is some discussion of this problem [8,38,61] but no certainty how best to approach this, and offering "advanced" tools (such as breakpoints, watches, and so on) brings a conceptual overhead that risks overwhelming novice learners.

Recommendation 5. Consideration for adequate support for debugging and troubleshooting of programs must be made in the design of environments for transition, but more investigation is needed to determine what the appropriate level of support is.

7.4. Evaluating efficacy

Finally, while this review has noted the diversity of efforts to evaluate approaches to supporting transition there is no doubt that further methodologically robust longitudinal evaluations of such interventions would be of enormous benefit to the community (and this need has also been noted in the literature, in e.g. [98,99])

Recommendation 6: Both new and existing approaches should seek to gather robust, empirical evidence to improve the evidence base for future designs.

8. Conclusion

This review has revealed that while a complete consensus on the essential elements of support for learners in the transition from BBP to TBP has yet to emerge there are common themes that can be identified. There needs to be further exploration of how best to integrate these themes to give learners the support that will encourage them to remain interested and engaged in computer science education. This review helps to identify the significant themes that efforts to support learners will have to engage with in order to further improve on the current state of the art, and has made a number of specific recommendations that should inform future development in the area.

CRediT authorship contribution statement

Glenn Strong: Writing – original draft, Formal analysis, Conceptualization. **Nina Bresnihan:** Writing – review & editing, Validation. **Brendan Tangney:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

Data will be made available on request.

References

[1] M. Resnick, J. Maloney, A. Monroy-Hernández, N. Rusk, E. Eastmond, K. Brennan, A. Millner, E. Rosenbaum, J. Silver, B. Silverman, Y. Kafai, Scratch: Programming for all, Commun. ACM 52 (11) (2009) 60–67, http://dx.doi.org/10.1145/1592761.1592779.

[2] K. Amanullah, T. Bell, Analysis of progression of scratch users based on their use of elementary patterns, in: 2019 14th International Conference on Computer Science & Education, ICCSE, IEEE, Toronto, ON, Canada, 2019, pp. 573–578, http://dx.doi.org/10.1109/ICCSE.2019.8845495, †.

[3] D. Weintrop, Block-based programming in computer science education, Commun. ACM 62 (8) (2019) 22–25, http://dx.doi.org/10.1145/3341221.

[4] C. Zdawczyk, K. Varma, Engaging girls in computer science: Gender differences in attitudes and beliefs about learning scratch and python, Comput. Sci. Educ. 33 (4) (2023) 600–620, http://dx.doi.org/10.1080/08993408.2022.2095593.

[5] M. Kölling, N.C.C. Brown, A. Altadmri, Frame-based editing: Easing the transition from blocks to text-based programming, in: Proceedings of the Workshop in Primary and Secondary Computing Education, ACM, London United Kingdom, 2015, pp. 29–38, http://dx.doi.org/10.1145/2818314.2818331, †.

[6] L. Moors, A. Luxton-Reilly, P. Denny, Transitioning from block-based to text-based programming languages, in: 2018 International Conference on Learning and Teaching in Computing and Engineering, LaTICE, IEEE, Auckland, New Zealand, 2018, pp. 57–64, http://dx.doi.org/10.1109/LaTICE.2018.000-5, †.

[7] K. Powers, S. Ecott, L.M. Hirshfield, Through the looking glass: Teaching CS0 with Alice, ACM SIGCSE Bull. 39 (1) (2007) 213–217, http://dx.doi.org/10.1145/1227504.1227386, †.

[8] M. Kazemitabaar, V. Chyhir, D. Weintrop, T. Grossman, Scaffolding progress: How structured editors shape novice errors when transitioning from blocks to text, in: Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1, ACM, Toronto ON Canada, 2023, pp. 556–562, http://dx.doi.org/10.1145/3545945.3569723, †.

[9] P. Kinnunen, L. Malmi, Why students drop out CS1 course? in: Proceedings of the Second International Workshop on Computing Education Research, ACM, Canterbury United Kingdom, 2006, pp. 97–108, http://dx.doi.org/10.1145/1151588.1151604.

[10] M. Armoni, O. Meerbaum-Salant, M. Ben-Ari, From scratch to "real" programming, ACM Trans. Comput. Educ. 14 (4) (2015) 1–15, http://dx.doi.org/10.1145/2677087, †.

[11] M. Noone, A. Mooney, Visual and textual programming languages: A systematic review of the literature, J. Comput. Educ. 5 (2) (2018) 149–174, http://dx.doi.org/10.1007/s40692-018-0101-5, †.

[12] A. Pears, S. Seidman, L. Malmi, L. Mannila, E. Adams, J. Bennedsen, M. Devlin, J. Paterson, A survey of literature on the teaching of introductory programming, ACM SIGCSE Bull. 39 (4) (2007) 204–223, http://dx.doi.org/10.1145/1345375.1345441.

[13] F.P. Deek, J.A. McHugh, A survey and critical analysis of tools for learning programming, Comput. Sci. Educ. 8 (2) (1998) 130–178, http://dx.doi.org/10.1076/csed.8.2.130.3820.

[14] M. Guzdial, Programming environments for novices, in: S. Fincher, M. Petre (Eds.), Computer Science Education Research, first ed., Taylor & Francis, 2004, pp. 127–154.

[15] R.P. Medeiros, G.L. Ramalho, T.P. Falcao, A systematic literature review on teaching and learning introductory programming in higher education, IEEE Trans. Educ. 62 (2) (2019) 77–90, http://dx.doi.org/10.1109/TE.2018.2864133.

[16] L. Sun, Z. Guo, D. Zhou, Developing K-12 students' programming ability: A systematic literature review, Educ. Inf. Technol. 27 (5) (2022) 7059–7097, http://dx.doi.org/10.1007/s10639-022-10891-2, †.

[17] Y. Lin, D. Weintrop, The landscape of block-based programming: Characteristics of block-based environments and how they support the transition to text-based programming, J. Comput. Lang. 67 (2021) 101075, http://dx.doi.org/10.1016/j.cola.2021.101075, †.

[18] D. Krpan, S. Mladenovic, G. Zaharija, Mediated transfer from visual to high-level programming language, in: 2017 40th International Convention on Information and Communication Technology, Electronics and Microelectronics, MIPRO, IEEE, Opatija, Croatia, 2017, pp. 800–805, http://dx.doi.org/10.23919/MIPRO.2017.7973531, †.

[19] D. Moher, A. Liberati, J. Tetzlaff, D.G. Altman, for the PRISMA Group, Preferred reporting items for systematic reviews and meta-analyses: The PRISMA statement, BMJ 339 (jul21 1) (2009) http://dx.doi.org/10.1136/bmj.b2535, b2535–b2535.

[20] J.P. Higgins, S. Green (Eds.), Cochrane Handbook for Systematic Reviews of Interventions: Cochrane Book Series, first ed., Wiley, 2008, http://dx.doi.org/10.1002/9780470712184.

[21] JBI Manual for Evidence Synthesis, JBI, 2020, http://dx.doi.org/10.46658/JBIMES-20-01.

[22] D. Gough, S. Oliver, J. Thomas, An introduction to systematic reviews (2nd edition), Psychol. Teach. Rev. 23 (2) (2017) 95–96, http://dx.doi.org/10.53841/bpsptr.2017.23.2.95.

[23] A. Liberati, D.G. Altman, J. Tetzlaff, C. Mulrow, P.C. Gøtzsche, J.P. Ioannidis, M. Clarke, P. Devereaux, J. Kleijnen, D. Moher, The PRISMA statement for reporting systematic reviews and meta-analyses of studies that evaluate health care interventions: Explanation and elaboration, J. Clin. Epidemiol. 62 (10) (2009) e1–e34, http://dx.doi.org/10.1016/j.jclinepi.2009.06.006.

[24] B. Kitchenham, Z. Li, A. Burn, Validating search processes in systematic literature reviews, in: Proceeding of the 1st International Workshop on Evidential Assessment of Software Technologies, SCITEPRESS - Science and Technology Publications, Beijing, China, 2011, pp. 3–9, http://dx.doi.org/10.5220/0003557000030009.

[25] B. Kitchenham, P. Brereton, A systematic review of systematic review process research in software engineering, Inf. Softw. Technol. 55 (12) (2013) 2049–2075, http://dx.doi.org/10.1016/j.infsof.2013.07.010.

[26] T. Kosar, S. Bohra, M. Mernik, A systematic mapping study driven by the margin of error, J. Syst. Softw. 144 (2018) 439–449, http://dx.doi.org/10.1016/j.jss.2018.06.078.

[27] H. Cai, G.K.W. Wong, A systematic review of studies of parental involvement in computational thinking education, Interact. Learn. Environ. (2023) 1–24, http://dx.doi.org/10.1080/10494820.2023.2214185.

[28] A. Luxton-Reilly, Simon, I. Albluwi, B.A. Becker, M. Giannakos, A.N. Kumar, L. Ott, J. Paterson, M.J. Scott, J. Sheard, C. Szabo, Introductory programming: A systematic literature review, in: Proceedings Companion of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education, ACM, Larnaca Cyprus, 2018, pp. 55–106, http://dx.doi.org/10.1145/3293881.3295779.

[29] M.M. McGill, A. Decker, Construction of a taxonomy for tools, languages, and environments across computing education, in: Proceedings of the 2020 ACM Conference on International Computing Education Research, ACM, Virtual Event New Zealand, 2020, pp. 124–135, http://dx.doi.org/10.1145/3372782.3406258.

[30] K. Nolan, S. Bergin, The role of anxiety when learning to program: A systematic review of the literature, in: Proceedings of the 16th Koli Calling International Conference on Computing Education Research, ACM, Koli Finland, 2016, pp. 61–70, http://dx.doi.org/10.1145/2999541.2999557.

[31] K.R. Felizardo, J.C. Carver, Automating systematic literature review, in: M. Felderer, G.H. Travassos (Eds.), Contemporary Empirical Methods in Software Engineering, Springer International Publishing, Cham, 2020, pp. 327–355, http://dx.doi.org/10.1007/978-3-030-32489-6_12.

[32] E. Syriani, I. David, G. Kumar, Screening articles for systematic reviews with ChatGPT, J. Comput. Lang. 80 (2024) 101287, http://dx.doi.org/10.1016/j.cola.2024.101287.

[33] C. Wohlin, M. Kalinowski, K. Romero Felizardo, E. Mendes, Successful combination of database search and snowballing for identification of primary studies in systematic literature studies, Inf. Softw. Technol. 147 (2022) 106908, http://dx.doi.org/10.1016/j.infsof.2022.106908.

[34] V. Clarke, Teaching thematic analysis: Victoria Clarke and Virginia Braun look at overcoming challenges and developing strategies for effective learning, Psychol. (2013).

[35] K. Olsen, P. Harnes, B. Pedersen, O.-J. Tosse, The DSP system-a visual system to support teaching of programming, in: [Proceedings] 1988 IEEE Workshop on Visual Languages, IEEE Comput. Soc. Press, Pittsburgh, PA, USA, 1988, pp. 199–206, http://dx.doi.org/10.1109/WVL.1988.18029, †.

[36] D. Weintrop, U. Wilensky, Between a block and a typeface: Designing and evaluating hybrid programming environments, in: Proceedings of the 2017 Conference on Interaction Design and Children, ACM, Stanford California USA, 2017, pp. 183–192, http://dx.doi.org/10.1145/3078072.3079715, †.

[37] Ž. Leber, M. Črepinšek, T. Kosar, Simultaneous multiple representation editing environment for primary school education, in: 2019 IEEE Symposium on Visual Languages and Human-Centric Computing, VL/HCC, IEEE, Memphis, TN, USA, 2019, pp. 175–179, http://dx.doi.org/10.1109/VLHCC.2019.8818927, †.

[38] C. Kyfonidis, N. Moumoutzis, S. Christodoulakis, Block-c: A block-based programming teaching tool to facilitate introductory c programming courses, in: 2017 IEEE Global Engineering Education Conference, EDUCON, IEEE, Athens, Greece, 2017, pp. 570–579, http://dx.doi.org/10.1109/EDUCON.2017.7942903, †.

[39] P. Bachiller-Burgos, I. Barbecho, L.V. Calderita, P. Bustos, L.J. Manso, Learn-Block: A robot-agnostic educational programming tool, IEEE Access 8 (2020) 30012–30026, http://dx.doi.org/10.1109/ACCESS.2020.2972410, †.

[40] Y. Lin, D. Weintrop, J. McKenna, Switch mode: A visual programming approach for transitioning from block-based to text-based programming, in: Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 2, ACM, Toronto ON Canada, 2022, http://dx.doi.org/10.1145/3545947.3573235, 1262–1262. †.

[41] Y. Lin, Switch mode: Exploring authoring python inside a block-based programming environment, in: 2023 IEEE Symposium on Visual Languages and Human-Centric Computing, VL/HCC, IEEE, Washington, DC, USA, 2023, pp. 312–313, http://dx.doi.org/10.1109/VL-HCC57772.2023.00064, †.

[42] D. Weintrop, U. Wilensky, How block-based, text-based, and hybrid block/text modalities shape novice programming practices, Int. J. Child Comput. Interact. 17 (2018) 83–92, †.

[43] A.C. Bart, E. Tilevich, C.A. Shaffer, D. Kafura, Position paper: From interest to usefulness with BlockPy, a block-based, educational environment, in: 2015 IEEE Blocks and beyond Workshop, Blocks and beyond), IEEE, Atlanta, GA, USA, 2015, pp. 87–89, http://dx.doi.org/10.1109/BLOCKS.2015.7369009, †.

[44] M. Bajzek, H. Bort, O. Hunpatin, L. Mivshek, T. Much, C. O'Hare, D. Brylow, MUzECS: Embedded blocks for exploring computer science, in: 2015 IEEE Blocks and beyond Workshop, Blocks and beyond), IEEE, Atlanta, GA, USA, 2015, pp. 127–132, http://dx.doi.org/10.1109/BLOCKS.2015.7369021, †.

[45] W. Dann, D. Cosgrove, D. Slater, D. Culyba, S. Cooper, Mediated transfer: alice 3 to java, in: Proceedings of the 43rd ACM Technical Symposium on Computer Science Education, ACM, Raleigh North Carolina USA, 2012, pp. 141–146, http://dx.doi.org/10.1145/2157136.2157180, †.

[46] A. Wagner, J. Gray, J. Corley, D. Wolber, Using app inventor in a K-12 summer camp, in: Proceeding of the 44th ACM Technical Symposium on Computer Science Education, ACM, Denver Colorado USA, 2013, pp. 621–626, http://dx.doi.org/10.1145/2445196.2445377, †.

[47] J.C. Cheung, G. Ngai, S.C. Chan, W.W. Lau, Filling the gap in programming instruction: A text-enhanced graphical programming environment for junior high students, in: Proceedings of the 40th ACM Technical Symposium on Computer Science Education, ACM, Chattanooga TN USA, 2009, pp. 276–280, http://dx.doi.org/10.1145/1508865.1508968, †.

[48] D. Bau, J. Gray, C. Kelleher, J. Sheldon, F. Turbak, Learnable programming: Blocks and beyond, Commun. ACM 60 (6) (2017) 72–80, http://dx.doi.org/10.1145/3015455, †.

[49] M. Homer, J. Noble, A tile-based editor for a textual programming language, in: 2013 First IEEE Working Conference on Software Visualization, VISSOFT, IEEE, Eindhoven, Netherlands, 2013, pp. 1–4, http://dx.doi.org/10.1109/VISSOFT.2013.6650546, †.

[50] M. Homer, J. Noble, Combining tiled and textual views of code, in: 2014 Second IEEE Working Conference on Software Visualization, IEEE, Victoria, BC, Canada, 2014, pp. 1–10, http://dx.doi.org/10.1109/VISSOFT.2014.11, †.

[51] D. Bau, D.A. Bau, M. Dawson, C.S. Pickens, Pencil code: Block code for a text world, in: Proceedings of the 14th International Conference on Interaction Design and Children, ACM, Boston Massachusetts, 2015, pp. 445–448, http://dx.doi.org/10.1145/2771839.2771875, †.

[52] D. Bau, M. Dawson, A. Bau, Using pencil code to bridge the gap between visual and text-based coding (abstract only), in: Proceedings of the 46th ACM Technical Symposium on Computer Science Education, ACM, Kansas City Missouri USA, 2015, http://dx.doi.org/10.1145/2676723.2678293, 706–706. †.

[53] M. Kazemitabaar, V. Chyhir, D. Weintrop, T. Grossman, CodeStruct: Design and evaluation of an intermediary programming environment for novices to transition from scratch to python, in: Interaction Design and Children, ACM, Braga Portugal, 2022, pp. 261–273, http://dx.doi.org/10.1145/3501712.3529733, †.

[54] J. Dillane, Frame-based novice programming, in: Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education, ACM, Trondheim Norway, 2020, pp. 583–584, http://dx.doi.org/10.1145/3341525.3394007, †.

[55] N. Brown, C. Kyfonidis, P. Weill-Tessier, B. Becker, J. Dillane, M. Kölling, A frame of mind: Frame-based vs. text-based editing, in: United Kingdom and Ireland Computing Education Research Conference, ACM, Glasgow United Kingdom, 2021, pp. 1–7, http://dx.doi.org/10.1145/3481282.3481286, †.

[56] T.W. Price, N.C. Brown, D. Lipovac, T. Barnes, M. Kölling, Evaluation of a frame-based programming editor, in: Proceedings of the 2016 ACM Conference on International Computing Education Research, ACM, Melbourne VIC Australia, 2016, pp. 33–42, http://dx.doi.org/10.1145/2960310.2960319, †.

[57] M. Dorling, D. White, Scratch: A way to logo and python, in: Proceedings of the 46th ACM Technical Symposium on Computer Science Education, ACM, Kansas City Missouri USA, 2015, pp. 191–196, http://dx.doi.org/10.1145/2676723.2677256, †.

[58] T. Tóth, G. Lovászová, Mediation of knowledge transfer in the transition from visual to textual programming, Inform. Educ. (2021) http://dx.doi.org/10.15388/infedu.2021.20, †.

[59] E. Dillon, M. Anderson, M. Brown, Comparing mental models of novice programmers when using visual and command line environments, in: Proceedings of the 50th Annual Southeast Regional Conference, ACM, Tuscaloosa Alabama, 2012, pp. 142–147, http://dx.doi.org/10.1145/2184512.2184546, †.

[60] C. Chen, P. Haduong, K. Brennan, G. Sonnert, P. Sadler, The effects of first programming language on college students' computing attitude and achievement: A comparison of graphical and textual languages, Comput. Sci. Educ. 29 (1) (2019) 23–48, http://dx.doi.org/10.1080/08993408.2018.1547564, †.

[61] P. Niemela, All rosy in scratch lessons: No bugs but guts with visual programming, in: 2017 IEEE Frontiers in Education Conference, FIE, IEEE, Indianapolis, IN, 2017, pp. 1–9, http://dx.doi.org/10.1109/FIE.2017.8190612, †.

[62] J. Sorva, Notional machines and introductory programming education, ACM Trans. Comput. Educ. 13 (2) (2013) 1–31, http://dx.doi.org/10.1145/2483710.2483713.

[63] K. Howland, J. Good, Learning to communicate computationally with flip: A bi-modal programming language for game creation, Comput. Educ. 80 (2015) 224–240, http://dx.doi.org/10.1016/j.compedu.2014.08.014, †.

[64] J. Monig, Y. Ohshima, J. Maloney, Blocks at your fingertips: Blurring the line between blocks and text in GP, in: 2015 IEEE Blocks and beyond Workshop, Blocks and beyond), IEEE, 2015, pp. 51–53, http://dx.doi.org/10.1109/BLOCKS.2015.7369001, †.

[65] D.C. Cliburn, Student opinions of alice in CS1, in: 2008 38th Annual Frontiers in Education Conference, IEEE, Saratoga Springs, NY, USA, 2008, http://dx.doi.org/10.1109/FIE.2008.4720254, T3B-1–T3B-6, †.

[66] D. Gudmundsen, L. Olivieri, N. Sarawagi, Using visual logic^{C}: Three different approaches in different courses - general education CS0, and CS1, J. Comput. Sci. Coll. 26 (6) (2011) 23–29, http://dx.doi.org/10.5555/1968521.1968529, †.

[67] R. Aish, E. Mendoza, DesignScript: A domain specific language for architectural computing, in: Proceedings of the International Workshop on Domain-Specific Modeling, ACM, Amsterdam Netherlands, 2016, pp. 15–21, http://dx.doi.org/10.1145/3023147.3023150, †.

[68] W. Robinson, From scratch to patch: Easing the blocks-text transition, in: Proceedings of the 11th Workshop in Primary and Secondary Computing Education, ACM, Münster Germany, 2016, pp. 96–99, http://dx.doi.org/10.1145/2978249.2978265, †.

[69] S. Xinogalos, M. Satratzemi, C. Malliarakis, Microworlds, games, animations, mobile apps, puzzle editors and more: what is important for an introductory programming environment? Educ. Inf. Technol. 22 (1) (2017) 145–176, http://dx.doi.org/10.1007/s10639-015-9433-1, †.

[70] H. Alrubaye, S. Ludi, M.W. Mkaouer, Comparison of block-based and hybrid-based environments in transferring programming skills to text-based environments, in: Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering, in: CASCON '19, IBM Corp., USA, 2019, pp. 100–109, †.

[71] G. Mecca, D. Santoro, N. Sileno, E. Veltri, Diogene-CT: Tools and methodologies for teaching and learning coding, Int. J. Educ. Technol. High. Educ. 18 (1) (2021) 12, http://dx.doi.org/10.1186/s41239-021-00246-1, †.

[72] N.C.C. Brown, M. Kölling, C. Kyfonidis, P. Weill-Tessier, Transitioning from blocks to text, in: Proceedings of the 53rd ACM Technical Symposium on Computer Science Education V. 2, ACM, Providence RI USA, 2022, pp. 1045–1046, http://dx.doi.org/10.1145/3478432.3499033, †.

[73] D. Weintrop, U. Wilensky, Transitioning from introductory block-based and text-based environments to professional programming languages in high school computer science classrooms, Comput. Educ. 142 (2019) 103646, http://dx.doi.org/10.1016/j.compedu.2019.103646, †.

[74] F. Alegre, J. Moreno, T. Dawson, E.E. Tanjong, D.H. Kirshner, Computational thinking for STEM teacher leadership training at louisiana state university, in: 2020 Research on Equity and Sustained Participation in Engineering, Computing, and Technology, IEEE, Portland, OR, USA, 2020, pp. 1–2, http://dx.doi.org/10.1109/RESPECT49803.2020.9272455, †.

[75] T. Toth, V. Michalickova, From app inventor to java: A strategy for mediating the transition, in: 2018 16th International Conference on Emerging ELearning Technologies and Applications, ICETA, IEEE, Starý Smokovec, 2018, pp. 591–596, http://dx.doi.org/10.1109/ICETA.2018.8572156, †.

[76] Y. Matsuzawa, Y. Tanaka, S. Sakai, Measuring an impact of block-based language in introductory programming, in: T. Brinda, N. Mavengere, I. Haukijärvi, C. Lewin, D. Passey (Eds.), Stakeholders and Information Technology in Education, vol. 493, Springer International Publishing, Cham, 2016, pp. 16–25, http://dx.doi.org/10.1007/978-3-319-54687-2_2, †.

[77] Y. Matsuzawa, T. Ohata, M. Sugiura, S. Sakai, Language migration in non-CS introductory programming through mutual language translation environment, in: Proceedings of the 46th ACM Technical Symposium on Computer Science Education, ACM, Kansas City Missouri USA, 2015, pp. 185–190, http://dx.doi.org/10.1145/2676723.2677230, †.

[78] A. Stead, A.F. Blackwell, Learning syntax as notational expertise when using DrawBridge, in: Annual Workshop of the Psychology of Programming Interest Group, 2014, pp. 41–52, †.

[79] D. Weintrop, N. Holbert, From blocks to text and back: Programming patterns in a dual-modality environment, in: Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education, ACM, Seattle Washington USA, 2017, pp. 633–638, http://dx.doi.org/10.1145/3017680.3017707, †.

[80] R. Garlick, E.C. Cankaya, Using alice in CS1: A quantitative experiment, in: Proceedings of the Fifteenth Annual Conference on Innovation and Technology in Computer Science Education, ACM, Bilkent Ankara Turkey, 2010, pp. 165–168, http://dx.doi.org/10.1145/1822090.1822138, †.

[81] A. Stead, How can we improve notational expertise? in: Proceedings of the Tenth Annual Conference on International Computing Education Research, ACM, Glasgow Scotland United Kingdom, 2014, pp. 173–174, http://dx.doi.org/10.1145/2632320.2632341, †.

[82] G. Strong, S. O'Carroll, N. Bresnihan, A block based editor for python, in: Proceedings of the 13th Workshop in Primary and Secondary Computing Education, ACM, Potsdam Germany, 2018, pp. 1–2, http://dx.doi.org/10.1145/3265757.3265788, †.

[83] G. Strong, B. North, Pytch — an environment for bridging block and text programming styles (work in progress), in: The 16th Workshop in Primary and Secondary Computing Education, in: WiPSCE '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 1–4, http://dx.doi.org/10.1145/3481312.3481318, †.

[84] D. Weintrop, U. Wilensky, Comparing block-based and text-based programming in high school computer science classrooms, ACM Trans. Comput. Educ. 18 (1) (2018) 1–25, http://dx.doi.org/10.1145/3089799, †.

[85] D. Gudmundsen, L. Olivieri, N. Sarawagi, Reducing the learning curve in an introductory programming course, J. Comput. Sci. Coll. 27 (3) (2012) 158–159, †.

[86] D. Gudmundsen, L. Olivieri, N. Sarawagi, Reducing the learning curve in an introductory programming course using visual logic[C], J. Comput. Sci. Coll. 27 (6) (2012) 10–12, †.

[87] G. Slomski, A. Jose Rohling, P. Varela, M. Albonico, HelloArduBot: A DSL for teaching programming to incoming students with open-source robotic (OSR) projects, in: The 18th International Symposium on Open Collaboration, ACM, Madrid Spain, 2022, pp. 1–5, http://dx.doi.org/10.1145/3555051.3555070, †.

[88] J. Rezgui, F. Jobin, S. Beaulieu, Z. Ardekani-Djoneidi, Autonomous learning intelligent vehicles engineering in a programming learning application for youth: ALIVE PLAY, in: 2021 International Symposium on Networks, Computers and Communications, ISNCC, IEEE, Dubai, United Arab Emirates, 2021, pp. 1–6, http://dx.doi.org/10.1109/ISNCC52172.2021.9615703, †.

[89] V. Dolgopolovas, T. Jevsikova, V. Dagiene, From android games to coding in C—An approach to motivate novice engineering students to learn programming: A case study, Comput. Appl. Eng. Educ. 26 (1) (2018) 75–90, http://dx.doi.org/10.1002/cae.21862, †.

[90] J. Strout, MiniScript: A new language for computer programming education, in: 2021 6th International STEM Education Conference, ISTEM-Ed, IEEE, Pattaya, Thailand, 2021, pp. 1–4, http://dx.doi.org/10.1109/iSTEM-Ed52129.2021.9625108, †.

[91] K. Umezawa, M. Nakazawa, K. Ishida, S. Hirasawa, Proposal and evaluation of intermediate content for the transition from visual to text-based programming languages, 56th Hawaii Int. Conf. Syst. Sci. (HICSS 2023) (2023) 83–92, †.

[92] I. Stripeikaitė, "Skipping the baby steps": The importance of teaching practical programming before programming theory, in: M. Alcañiz, S. Göbel, M. Ma, M. Fradinho Oliveira, J. Baalsrud Hauge, T. Marsh (Eds.), Serious Games, vol. 10622, Springer International Publishing, Cham, 2017, pp. 319–330, http://dx.doi.org/10.1007/978-3-319-70111-0_30, †.

[93] K. Dilmen, S.B. Kert, T. Uğraş, Children's coding experiences in a block-based coding environment: A usability study on code.org, Educ. Inf. Technol. 28 (9) (2023) 10839–10864, http://dx.doi.org/10.1007/s10639-023-11625-8, †.

[94] F. Demir, The effect of different usage of the educational programming language in programming education on the programming anxiety and achievement, Educ. Inf. Technol. 27 (3) (2022) 4171–4194, http://dx.doi.org/10.1007/s10639-021-10750-6, †.

[95] K. Umezawa, Y. Ishii, M. Nakazawa, M. Nakano, M. Kobayashi, S. Hirasawa, Comparison experiment of learning state between visual programming language and text programming language, in: 2021 IEEE International Conference on Engineering, Technology & Education, TALE, IEEE, Wuhan, Hubei Province, China, 2021, pp. 01–05, http://dx.doi.org/10.1109/TALE52509.2021.9678608, †.

[96] K. Umezawa, T. Koshikawa, M. Nakazawa, S. Hirasawa, Differential analysis of heart rate, facial expressions and brain wave during learning of visual- and text-based languages, in: 2024 IEEE World Engineering Education Conference, EDUNINE, IEEE, Guatemala City, Guatemala, 2024, pp. 1–6, http://dx.doi.org/10.1109/EDUNINE60625.2024.10500611, †.

[97] G. Sim, M. Lochrie, M.S. Zubair, O. Kerr, M. Bates, Moving away from the blocks: evaluating the usability of EduBlocks for supporting children to transition from block-based programming, in: Human-Computer Interaction – INTERACT 2023: 19th IFIP TC13 International Conference, York, UK, August 28 – September 1, 2023, Proceedings, Part I, Springer-Verlag, Berlin, Heidelberg, 2023, pp. 317–336, http://dx.doi.org/10.1007/978-3-031-42280-5_19, †.

[98] F. Turbak, Taking stock of blocks: Promises and challenges of blocks programming languages, in: 2015 IEEE Symposium on Visual Languages and Human-Centric Computing, VL/HCC, IEEE, Atlanta, GA, 2015, http://dx.doi.org/10.1109/VLHCC.2015.7356971, 2–2. †.

[99] J. Blanchard, Hybrid environments: A bridge from blocks to text, in: Proceedings of the 2017 ACM Conference on International Computing Education Research, ACM, Tacoma Washington USA, 2017, pp. 295–296, http://dx.doi.org/10.1145/3105726.3105743, †.